Cross-System Data Drift & Trust Monitoring Platform

Final Project

CELEBAL EXCELLENCE INTERSHIP

Tech Stack: Python | PySpark | SQL | Databricks | Delta Lake

Abstract

Modern enterprises operate across multiple interconnected systems — CRM, Billing, Analytics — that update independently and drift out of sync without warning. This project presents a Cross-System Data Drift & Trust Monitoring Platform: a Databricks-native pipeline, built on a Medallion (Bronze/Silver/Gold) Lakehouse architecture, that detects cross-system record mismatches, quantifies three distinct forms of data drift, computes a weighted Data Trust Score per source and platform-wide, and raises configurable, severity-tiered alerts. The system was implemented end-to-end in PySpark and Delta Lake, validated against synthetic data with deliberately injected drift, and confirmed to correctly detect every injected anomaly with zero execution errors.

1. Introduction

Enterprises rarely have one authoritative source of truth. A customer's record lives in a CRM, their transactions live in a Billing system, and their aggregated behavior is reported by an Analytics platform — three independently-maintained views of the same underlying reality. When these views disagree, or when any one of them silently changes shape over time, the business consequences range from bad reporting to real financial and compliance risk. This platform exists to make that disagreement and change observable, quantified, and actionable, rather than something discovered after the fact.

2. Problem Statement

Independent update cycles and integration failures between CRM, Billing, and Analytics systems produce three categories of problems:

- Cross-system disagreement — conflicting, missing, or duplicate records between systems that should logically agree.
- Silent pipeline drift — schema changes, volume anomalies, or distributional shifts introduced by upstream changes, with no centralized mechanism to detect them.
- No unified trust signal — no single, continuously-computed measure of “how much should we trust this data right now,” forcing every downstream consumer to independently assess data quality.

There is currently no automated, scalable solution that continuously compares these systems, detects drift, and produces a trust signal in near real time.

3. Theoretical Background

3.1 Drift vs. Trust — Two Distinct Failure Modes

- Drift is a temporal comparison: does a system’s data today look like it did at a prior reference point (schema, volume, distribution)?
- Cross-system disagreement is a cross-sectional comparison: do two systems agree with each other right now?

A platform addressing only one of these is incomplete — persistent disagreement that never changes wouldn’t register as “drift,” and drift within a single system wouldn’t register as “disagreement.” This project implements both, unified by a third construct: the Trust Score.

3.2 Comparison Logic as Set Theory

Cross-system comparison reduces to operations over sets of keys:

- Missing Records = CRM_keys − Billing_keys (relative complement) — implemented as a left_anti join. Measures referential completeness.
- Extra Records = Billing_keys − CRM_keys — the same operation reversed. Measures referential integrity; orphan records indicate sync failures or fabricated references.
- Aggregation Mismatch — for keys present in both systems, compare independently-aggregated totals, flagged when the percentage difference exceeds a tolerance τ.

```
pct_diff = |billing_total − analytics_total| / |billing_total|
```

A nonzero tolerance is theoretically necessary — rounding and timing differences between pipelines produce legitimate small disagreement that isn't a real trust problem. This check requires both systems to be aggregated to the same grain (e.g. both per-customer, or both re-aggregated to per-day) before comparison is meaningful.

3.3 Drift Detection Theory

Type	What it catches	Method
Schema Drift	Structural contract violations — added/removed/type-changed columns	Diff {column: type} between baseline and current
Volume Drift	Abnormal change in record count	(current − baseline) / baseline, flagged past a % threshold
Distribution Drift	Shape change in a numeric field, even at constant volume	Population Stability Index (PSI)

PSI formula:

```
PSI = Σ (current_pct_i − baseline_pct_i) × ln(current_pct_i / baseline_pct_i)
```

computed over matching bins of baseline vs. current values. PSI is a symmetrized approximation of KL divergence, and is shape-sensitive in a way that mean/stddev comparisons are not — two distributions can share a mean while their underlying shape (e.g. bimodality) diverges completely.

PSI Range	Interpretation
< 0.10	No significant shift
0.10 – 0.25	Moderate shift — investigate
> 0.25	Major shift — likely real change

3.4 Trust Scoring as a Weighted Composite

```
trust_score = w1·completeness + w2·consistency + w3·volume_stability + w4·schema_stability +
w5·distribution_stability
```

A weighted composite is used rather than a simple average (which would treat all failure modes as equally severe) or a hard pass/fail gate (too brittle — a single moderate PSI shift shouldn't zero out an otherwise-healthy score). Weights in this implementation (completeness=0.30, consistency=0.30, volume=0.15, schema=0.15, distribution=0.10) prioritize directly-observable data-quality facts (nulls, mismatches) over drift signals, which are directional risk indicators rather than definitive quality judgments.

3.5 Alerting Theory

Fixed, configurable thresholds (rather than statistical/ML anomaly detection) are used deliberately for this stage of the platform: interpretability (a threshold breach is immediately explainable), independence from historical baselines (thresholds work from day one, before enough history exists to train an anomaly model), and auditability (defensible to a compliance reviewer). Two-tier severity (WARNING / CRITICAL) allows differentiated downstream routing — dashboard visibility vs. active paging. ML-based anomaly detection is explicitly future scope (Section 8), to be layered on once sufficient Gold-layer history accumulates.

4. Proposed System Architecture

The platform follows a Medallion (Lakehouse) Architecture:

```
Bronze (raw truth, batch + streaming)
  |
  v
Silver (standardized, comparable, deduplicated)
  |
  +--> Comparison Logic --> completeness/consistency --+
  |                                                    |
  +--> Drift Detection ---> stability metrics ----------+--> Trust Score --> Alerts --> Gold
  |                                                    |
  +----------------------------------------------------+
```

- Bronze — CRM, Billing, and Analytics ingested as-is (batch), plus billing events ingested via Spark Structured Streaming, preserving raw fidelity for reproducibility.
- Silver — deduplication, null-key removal, type casting, string normalization; streamed events merged into the current billing set.
- Gold — every derived result (missing/extra records, aggregation mismatches, drift report, trust scores, alerts) persisted as its own Delta table for downstream BI consumption.

This layering provides a replay boundary: scoring-logic changes only require re-running Silver→Gold; cleaning-logic changes only require re-running Bronze→Gold — source systems need not be re-queried.

5. Implementation Summary

Proposal Requirement	Implemented As
Batch ingestion	spark.read.csv(...) per source, per snapshot (baseline + current)
Streaming ingestion	Spark Structured Streaming, trigger(availableNow=True), into bronze_billing_stream
Cleaning/standardization	dropDuplicates, null-key removal, trim/lower/upper, to_date casting
Missing / Extra Records	left_anti joins, both directions
Aggregation Mismatch	Per-key grouped sums compared across systems within a % tolerance
Schema / Volume / Distribution Drift	Column-set diff / % count change / PSI, each vs. a distinct baseline snapshot
Trust Scoring	Weighted composite, computed per-source and platform-wide
Alerting	Threshold-driven, severity-tiered (CRITICAL/WARNING/INFO), Gold-persisted
Gold persistence	6 result tables: missing customers, extra records, aggregation mismatches, drift report, trust scores, alerts

---

6. Results & Validation

The pipeline was executed end-to-end against synthetic data (~29,000 records across CRM/Billing/Analytics) with deliberately injected drift — a new payment_channel column, a 28% billing volume spike, a 35% transaction-amount distribution shift, and orphan billing records — to confirm the detectors actually fire rather than merely being present in code. Validated output:

```
Missing Records (CRM customers, no billing) :   4599
Extra Records   (orphan billing rows)       :   1500
Aggregation Mismatches (> 2% diff)          :    789 / 6134 compared customers

Schema Drift  [billing]    : DETECTED (added=['payment_channel'])
Volume Drift  [billing]    : +28.0%  DETECTED
Volume Drift  [analytics]  : +30.5%  DETECTED
Distribution Drift [billing.amount] : PSI=0.2535  DETECTED

Trust Scores : crm=90.76, billing=56.5, analytics=76.5, platform=66.31
Alerts Raised: 9 (2 CRITICAL, 6 WARNING, 1 INFO)
```

All 19 Bronze/Silver/Gold Delta tables were created successfully with zero execution errors across every pipeline stage.

7. Conclusion

Data trust is foundational to reliable analytics and business decision-making. This platform demonstrates that cross-system drift and trust monitoring can be fully automated on a modern Lakehouse stack: every requirement in the original proposal — batch and streaming ingestion, three-way comparison logic, three distinct drift-detection techniques, a weighted trust-scoring framework, and configurable alerting — is implemented, theoretically grounded, and empirically validated against data with known injected anomalies.

8. Future Scope

- AI-based anomaly detection — replacing/augmenting fixed thresholds with models trained on accumulated Gold-layer history (e.g. isolation forests over drift metrics), once sufficient history exists.
- Automated data correction — reconciliation workflows that act on detected mismatches rather than only reporting them.
- Multi-cloud deployment — generalizing beyond the current Databricks/Delta-specific implementation.